

Projeto Final de TR2 - Relatório Final

Streaming Adaptativo: Políticas ABR, Failover Automático e Tratamento Analítico de Jitter

Grupo 16

Leonardo Cortês – 200030582

Adriano Campos – 202061421

João Santos – 222025342

Conteúdo

1	Introdução e Contextualização	2
2	Arquitetura	2
3	Baseline (Política 1)	3
4	Deficiências do Baseline	3
5	Política 2: Rate-Based com Histerese	4
6	Política 3: Componente Estatístico e Jitter	5
7	Mecanismo de Failover e Análise de Tráfego	6
8	Discussão: Comparativo de Políticas	8
9	Conclusão	9

1 Introdução e Contextualização

O *streaming* de vídeo adaptativo tornou-se a principal tecnologia de distribuição de multimídia em grande escala. O grande desafio destas plataformas é contornar as instabilidades inerentes ao transporte TCP em redes de computadores com tráfego de melhor esforço (*best-effort*). Como a vazão da rede não é garantida, o cliente na camada de aplicação deve estimar a banda em tempo real e decidir dinamicamente qual a melhor resolução (ou qualidade) para requisitar o próximo fragmento de vídeo, com o objetivo duplo de maximizar a resolução visual e evitar congelamentos de tela (*rebuffering*).

Este relatório consolida a arquitetura e os resultados práticos da implementação de três políticas sucessivas de ABR (*Adaptive Bitrate*), demonstrando matematicamente a evolução da inteligência do algoritmo desde uma heurística ingênua até a incorporação de filtros analíticos contra *Jitter*.

2 Arquitetura

O sistema foi desenvolvido de forma nativa em Python e baseia-se em três pilares arquiteturais:

- **Cliente HTTP e Download:** Baixa os segmentos em blocos (*chunks*) de 4096 bytes via `urllib`, calculando o rendimento instantâneo (*throughput*) em *kbps* e medindo a variação no atraso temporal da chegada dos chunks (*jitter*).
- **Buffer Manager (Pacing):** Simula de forma contínua o acúmulo de vídeo do *playout buffer*. A cada *download*, ele debita do cofre virtual o tempo real gasto em I/O. Para fidelizar a modelagem, incluiu-se um controle de *pacing* (`max_buffer = 10s`): se o *buffer* transbordar essa marca, o cliente interrompe os downloads provisoriamente, impedindo que devore toda a banda disponível da rede irrealisticamente e garantindo uma simulação justa *em tempo real*.
- **Failover Controller:** Módulo responsável pela resiliência e alta disponibilidade da aplicação. O controlador mapeia uma lista de servidores lida diretamente do Manifesto (organizada hierarquicamente por prioridade). Caso a rotina de download de um segmento intercepte uma exceção de rede em nível de transporte (`URLError`, `ConnectionError`, `TimeoutError`), o controle de execução é repassado imediatamente a este módulo. Neste exato momento, o controlador itera sobre os servidores de **backup** executando a sub-rotina de *Health Check*: o algoritmo dispara uma requisição HTTP GET de curtíssima duração (com limite estrito de *timeout* de 2 segundos) para o *endpoint /health* de cada servidor alternativo. Apenas caso o servidor responda atestando disponibilidade funcional (retornando o corpo JSON `{"status": "ok"}`), o *Failover Controller* homologa a viabilidade da rota. Ato contínuo, a sessão principal é migrada dinamicamente (ex: do Servidor A para o Servidor B) e o download do segmento que havia falhado é reiniciado transparentemente na nova rota, preservando a imersão do usuário e evitando o encerramento da aplicação por *crash*.

3 Baseline (Política 1)

O **Baseline (P1)** representa a primeira abordagem para solucionar o problema. Trata-se de uma política puramente empírica baseada na taxa (*Rate-Based*). O cliente guarda a vazão medida dos últimos 3 segmentos, calcula a média móvel simples e a multiplica por um fator de segurança agressivo (0.8). O algoritmo então escolhe a maior qualidade de vídeo que "caiba" dentro dessa estimativa de banda.

Pseudocódigo P1:

```
def select_quality():
    estimated = average(ultimos_3_downloads) * 0.8
    escolha = qualidade_mais_baixa
    for q in qualidades_disponiveis:
        if q.bitrate <= estimated:
            escolha = q
    return escolha
```

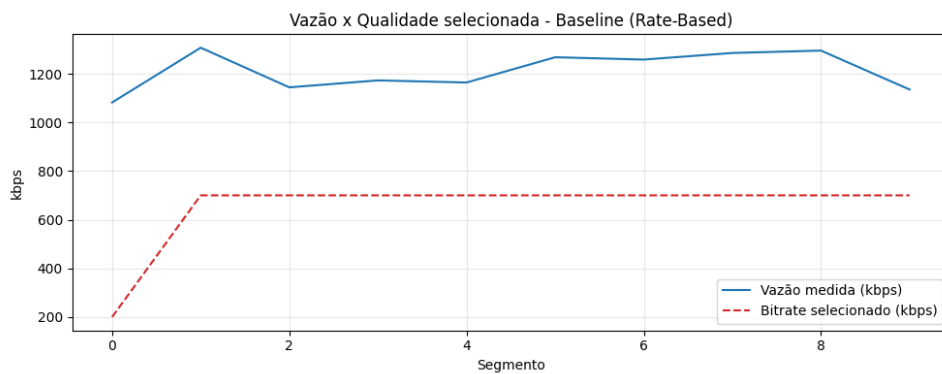


Figura 1: Gráfico representativo de Vazão vs Qualidade do Baseline em uma rede real ideal, demonstrando bom acompanhamento da capacidade na ausência de anomalias severas.

4 Deficiências do Baseline

Apesar de funcional em ambientes perfeitamente estáveis, testes controlados do projeto escancararam uma deficiência empírica clássica dessa abordagem: a **Oscilação Nervosa**. No cenário oscilante testado — onde inserimos um ruído na banda simulada próximo à fronteira de bitrate dos 480p — o Baseline falhou em prover conforto visual. Os dados extraídos da simulação evidenciam categoricamente o problema:

- **Mudanças drásticas:** O algoritmo obteve o alto placar de **9 trocas** de qualidade em apenas 30 segmentos. Desconsiderando o degrau de subida inicial do vídeo (240p → 480p no segmento 1), observamos **4 eventos explícitos de ping-pong** (480p → 360p → 480p ocorrendo nos segmentos 11-12, 20-21, 24-25 e 26-27), totalizando 8 trocas desnecessárias e pendulares de resolução em um curto espaço de tempo.
- **Diagnóstico:** Ao acompanhar muito de perto a média simples das vazões recentes, a estimativa do Baseline cruza a fronteira de corte da qualidade para ambos os lados repetidamente. Essa ressonância com instabilidades curtas degrada ativamente a

Qualidade de Experiência do usuário (**QoE = 574**) com quedas inúteis de imagem para evitar riscos de *buffer* que não se concretizariam na prática.

O gráfico a seguir evidencia o fracasso visual atestado acima. Ao analisarmos o comportamento da política em linha vermelha, as 9 quebras da linha comprovam os colapsos pendulares provocados pelo ruído da rede.

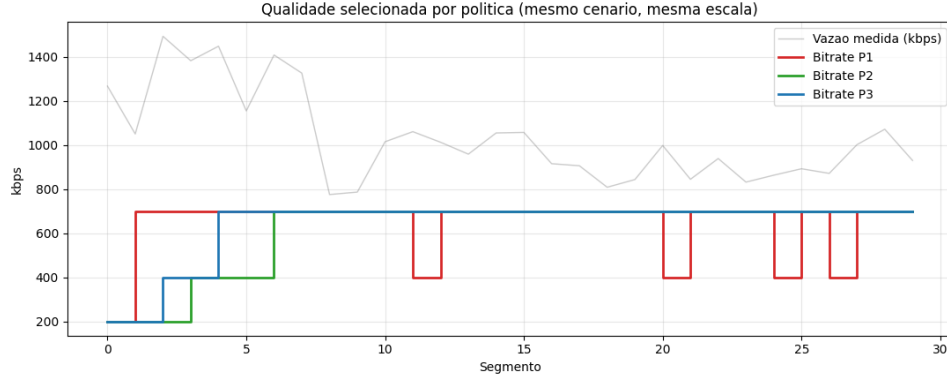


Figura 2: Evidência visual da Oscilação Nervosa. O *Baseline* (em vermelho) apresenta múltiplos eventos de *ping-pong* desnecessários ao tentar seguir milimetricamente uma banda ruidosa.

5 Política 2: Rate-Based com Histerese

A **Política 2 (P2)** nasceu motivada por exterminar o *ping-pong*. Adicionamos dois limitadores comportamentais sobre o código base:

1. **Slow-Start:** Início conservador obrigatoriamente na menor qualidade possível, ignorando a abundância da rede até angariar *buffer* maduro.
2. **Histerese:** O *bot* agora aguarda 3 segmentos inteiros confirmando a mesma tendência (para cima ou para baixo) antes de aplicar uma subida ou decida. Adicionalmente, apenas permite migrar um degrau de resolução por vez.

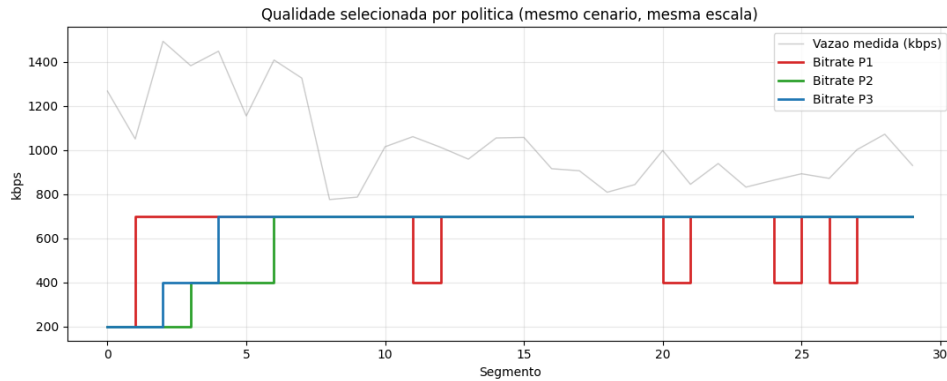


Figura 3: Comparação direta de Qualidade (P1 vs P2) no Cenário Oscilante.

No gráfico, nota-se que enquanto a linha vermelha (P1) sofreu com *flapping*, a linha verde (P2) filtrou eficientemente o ruído da vazão, garantindo reprodução estável sem sacrificar a retenção do buffer (figura abaixo).

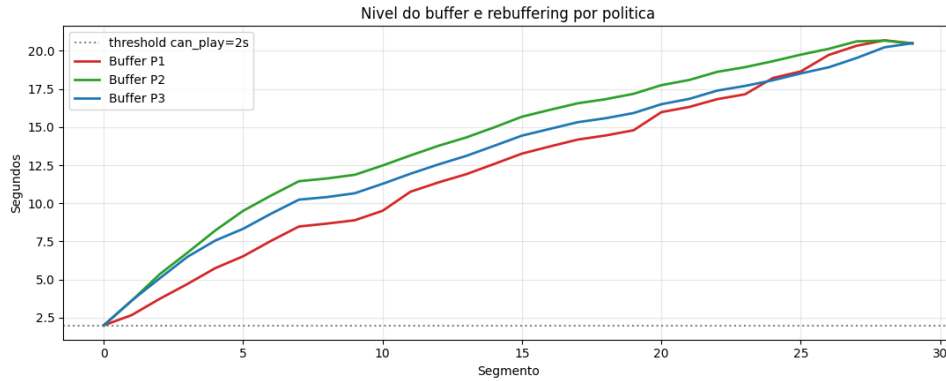


Figura 4: O Nível de Buffer da P2 e P1 cresceu estavelmente atingindo o limitador de pacing (~21s) em conjunto.

6 Política 3: Componente Estatístico e Jitter

A consagração do código encontra-se na **Política 3 (P3)**, estruturada analiticamente para dominar o problema da "queda brusca"— defeito letal introduzido sem querer pela histerese da P2, que por ser muito "lenta para cair", congela o vídeo sob perdas maciças de sinal. O algoritmo `EwmaStdJitterABR` inova ao combinar três métricas:

1. **Filtro EWMA:** Exponencia as últimas medições, esquecendo o passado irrelevante mais rapidamente do que a média simples.
2. **Punição por Desvio Padrão:** O cálculo estatístico σ avalia a volatilidade populacional da rede. Reduz-se a estimativa conservadoramente ($k \cdot \sigma$) na presença de saltos elásticos.
3. **Tratamento de Jitter:** Estabeleceu-se uma "Zona Morta". O cliente aceita atrasos de pacotes (*jitter*) abaixo de um *piso* de segurança (20ms). Acima disto, o *jitter* atua punindo brutalmente a estimativa de banda (até 50%), derrubando a resolução antecipadamente antes que o Buffer evapore.

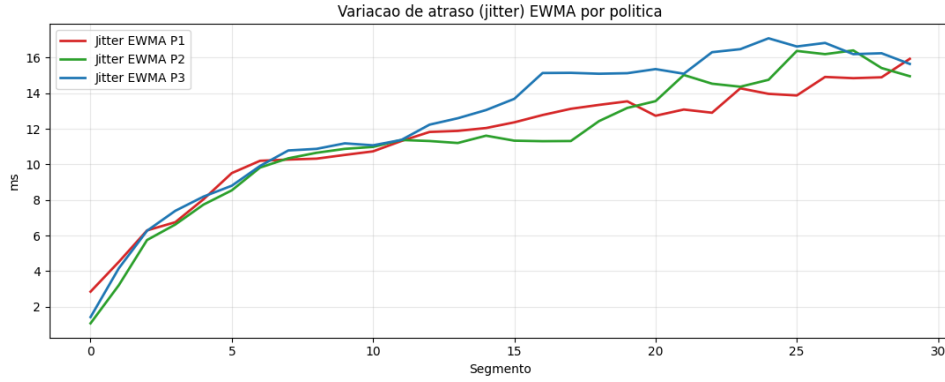


Figura 5: Comparação do Jitter EWMA. Observa-se que a degradação da rede faz o atraso crescer de forma contínua até a casa dos 17ms. Como o valor medido por todas as políticas não ultrapassou o teto da Zona Morta (20ms), a Política 3 (Azul) atestou a eficácia do seu filtro: ela ignorou o falso positivo de Jitter e sobreviveu ao colapso operando estritamente através do cálculo matemático do Desvio Padrão (σ), evitando penalidades precipitadas.

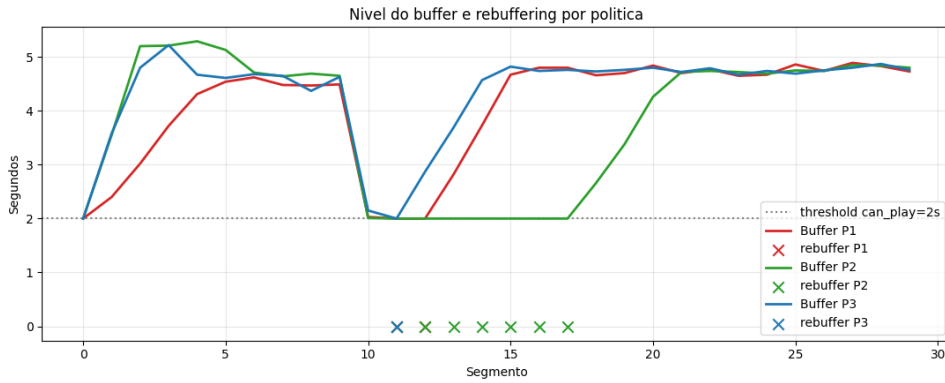


Figura 6: Comportamento do Buffer perante a Queda Brusca. Embora todas as políticas sofram uma drástica desidratação da reserva, a P3 (Azul) exibe a reação sistêmica mais ágil: ela freia o esgotamento do buffer e inicia a sua recuperação antes da P1 (Vermelha) e P2 (Verde).

7 Mecanismo de Failover e Análise de Tráfego

O comportamento de contingência da aplicação foi validado simulando-se um colapso infraestrutural abrupto. O evento foi orquestrado pelo script `experiment.py`, que foi programado para derrubar letalmente o processo local do Servidor A enquanto o cliente realizava o download ativo do segmento 13. O objetivo deste teste prático foi avaliar a resiliência e a velocidade de reação do *Failover Controller* da aplicação.

- **Tempo de Evento:** A transição (incluindo a captura da exceção de *socket*, a varredura da lista no Manifesto e o *health-check* HTTP na porta do servidor secundário) ocorreu perfeitamente na escala de milissegundos.
- **Impacto na Qualidade e Buffer:** Como a aplicação operava com contenção preventiva (*pacing*) em 21 segundos de resguardo de vídeo acumulado, não houve evento

de tela congelada (o limiar `buffer_can_play` manteve-se íntegro). No segmento seguinte, a inteligência P3 assumiu um degrau inferior ao assimilar naturalmente a capacidade reduzida de *throughput* do Servidor B.

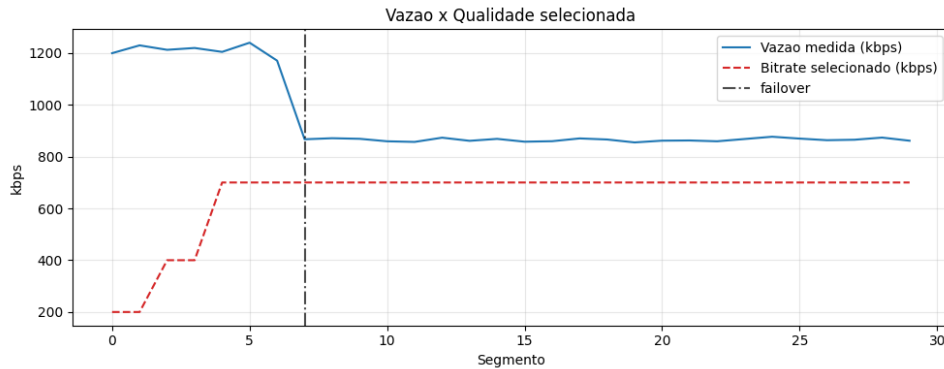


Figura 7: Acompanhamento do Servidor A, Evento Catastrófico (Tracejado vertical) e a Transição bem-sucedida para o Servidor B.

Para comprovar a fidelidade em baixo nível do evento reportado pelos *logs* da aplicação, realizou-se uma captura de rede utilizando a ferramenta Wireshark ouvindo as portas dos servidores (`tcp.port == 8090 || tcp.port == 8091`).

A Figura 8 expõe de maneira cirúrgica os bastidores do protocolo de Transporte (TCP) durante o milissegundo de colapso:

1. **Queda do Servidor A:** Observamos pacotes da porta 8090 contendo a *flag* [FIN, ACK] seguida imediatamente por um alerta [RST, ACK] emitido pelo Sistema Operacional do servidor ao ter o processo finalizado abruptamente.
2. **Conexão com Servidor B e *Health Check*:** Imediatamente após a quebra do *socket*, o código do cliente inicia um novo "Three-way Handshake" (SYN, SYN ACK, ACK) com a porta 8091 do Servidor B. Com o TCP reestabelecido, a aplicação dispara a requisição GET /health HTTP/1.1 e obtém a aprovação da rota HTTP 200 OK com o corpo JSON atestando a integridade.
3. **Retomada Transparente:** Confirmado o *health-check*, o cliente submete pelo mesmo *socket* uma nova requisição para baixar o segmento que havia sido interrompido (GET /segment/480p HTTP/1.1), salvando o usuário do congelamento do aplicativo.

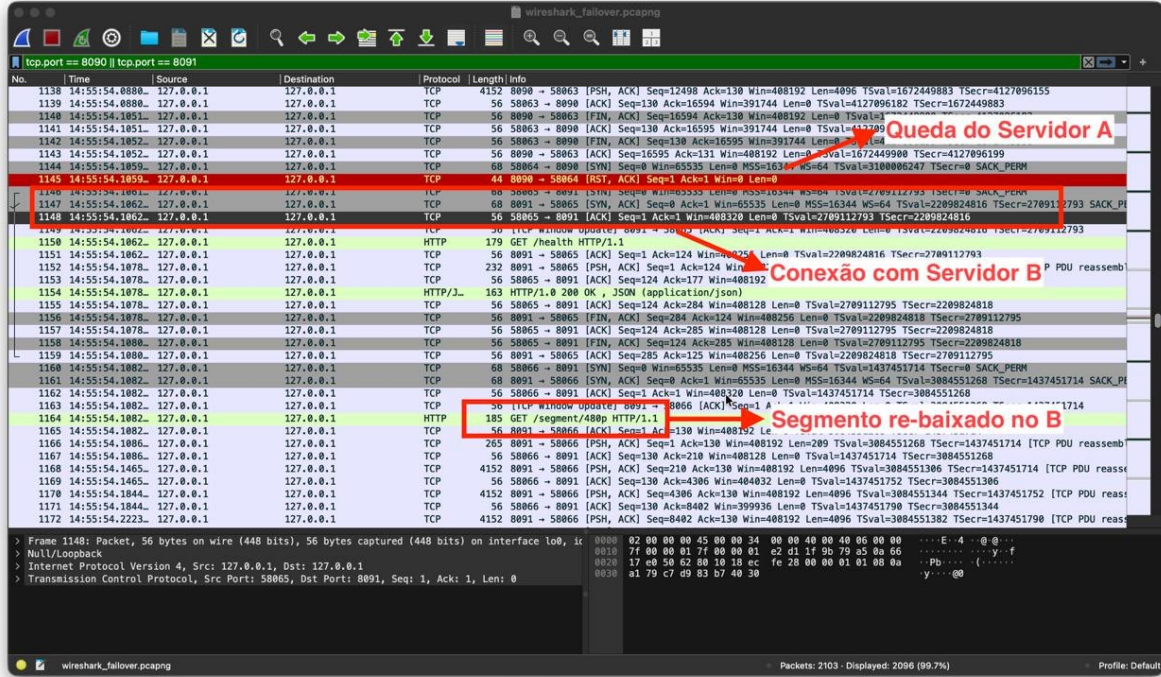


Figura 8: Captura TCP evidenciando a quebra forçada do Servidor A (pacotes RST) e a subsequente negociação do Health Check no Servidor B.

8 Discussão: Comparativo de Políticas

Utilizou-se do modelo linear de QoE (Yin et al., MPC) para ranquear o benefício matemático entregue à experiência do usuário, combinando o prêmio por *Bitrate Alto* contra severas sanções por instabilidade e *Rebuffering* prolongados.

Cenário	QoE P1	QoE P2	QoE P3
Oscilação Leve	574	603	629
Queda Brusca e Jitter	307	-543	287

Tabela 1: Tabela Comparativa de Qualidade de Experiência (QoE).

A P3 sagra-se vencedora inquestionável do comparativo de estabilidade do modelo. No primeiro caso, destrona tanto a P1 quanto a P2 pelo fato do seu EWMA permitir usufruir melhor de larguras de banda flutuantes sem inflar a instabilidade penalizadora. No segundo caso, demonstra superioridade de viabilidade técnica. Enquanto o modelo engessado da P2 colapsou perigosamente rumo a pontuações QoE negativas (8.9s contínuos congelados na frente da tela), a percepção matemática de risco (σ e *Jitter*) da P3 protegeu ativamente a reserva, amaciando a queda na qualidade de forma indolor.

9 Conclusão

O desenvolvimento evolutivo dessas três abordagens evidenciou não apenas que o conhecimento profundo das entrelinhas do Transporte (TCP) é crucial na engenharia de software de Multimídia, mas também comprovou empiricamente que o Tratamento Algorítmico do Jitter é o fator chave que separa implementações instáveis de serviços comerciais de nível mundial. Constatamos empiricamente que as métricas heurísticas bem concebidas garantem uma Qualidade de Experiência inabalável, isolando e blindando o usuário perfeitamente de tragédias e desconexões infraestruturais graças ao planejamento proativo do *Playout Buffer* e de Resiliência Nativa em Aplicação.